

05_m10_m12

M10 - Evaluation Harness & CI (L2 EXIT)

Objective

Prove the central empirical claim - **root-cause segment accuracy $\geq 85\%$ vs $\leq 45\%$ for the naive baseline**, with **0 hallucinated columns** - on an offline, *labeled* benchmark, and wire it into GitHub Actions as a hard regression firewall. The benchmark works by injecting synthetic-but-known anomalies into a frozen clone of the GA4 data (`helios_eval.fct_daily_funnel_perturbed`), running the full 7-agent FSM against the perturbed copy, and grading its diagnosis against ground truth recorded at injection time. This is the L2 exit: once green, no future change may drop top-1 accuracy more than `regression_tolerance` or introduce any hallucination. Grade STATISTICALLY (aggregate scores), never on byte-identical LLM output. See **Bible section 20** (Evaluation Framework) end to end.

Files to create

Path	Copy from
<code>helios/eval/injector.py</code>	Bible section 20.2 (injection mechanism: rate vs volume/mix primitives, seeded, writes <code>labels</code>)
<code>helios/eval/runner.py</code>	Bible section 20.6 (the <code>run_benchmark</code> core loop)
<code>helios/eval/report.py</code>	Bible section 20.9 (results-table template) + per-scenario JSON
<code>helios/eval/scorers/rootcause.py</code>	Bible section 20.4 / 20.7 (top-1 + top-3 on normalized segment key)
<code>helios/eval/scorers/decomposition.py</code>	Bible section 20.4 (MAPE on mix/rate/interaction, controls excluded)
<code>helios/eval/scorers/detection.py</code>	Bible section 20.4 (precision/recall/F1 over <code>(scenario, day, metric)</code> cells)
<code>helios/eval/scorers/dollars.py</code>	Bible section 20.4 (abs % error vs label <code>dollar_at_risk_usd</code>)

Path	Copy from
<code>helios/eval/scorers/hallucination.py</code>	Bible section 20.4 (SQL AST vs semantic-mcp registry + GA4 schema; hard zero)
<code>helios/eval/scorers/faithfulness.py</code>	Bible section 20.7 (two checks: numeric-claim hash rule + Critic-as-judge entailment)
<code>eval/gates.yaml</code>	Bible section 20.8 (thresholds, below)
<code>eval/baselines/main.json</code>	committed <code>main</code> -branch scores (seed from your first green full run)
<code>.github/workflows/ci.yaml</code>	Bible section 20.8 (dbt build + tests + smoke-on-push / full-on-PR gate)
<code>eval/scenarios/*.yaml</code>	ALREADY EXISTS - 50 labeled scenarios across 7 buckets (<code>scenarios.yaml</code> + the 7 bucket files). Do not regenerate.

`eval/gates.yaml` (verbatim from Bible section 20.8):

```
gates:
  rootcause_top1_min: 0.85
  hallucination_rate_max: 0.00      # hard zero
  decomposition_nape_max: 0.10
  detection_fl_min: 0.85
  dollar_error_max: 0.15
  faithfulness_min: 0.95
  regression_tolerance: 0.02      # top-1 may not drop >2pts vs main baseline
```

The shipped benchmark is **50 scenarios across 7 buckets** (the live `eval/scenarios/` directory): `single_segment_rate` (10), `single_segment_mix` (10), `multi_segment_rate` (6), `multi_segment_mixed` (6), `seasonality_decoy` (6), `no_anomaly_control` (6), `data_quality` (6). The decoy and control buckets are what separate Helios from the baseline - they punish over-flagging.

Commands to run

```
# POSIX: source .venv/bin/activate | Windows: .venv\Scripts\activate

# 0) materialize the frozen baseline clone the injector perturbs (one-time)
python -m helios.eval.injector --build-base \
  --src helios.marts.fct_daily_funnel \
  --dst helios_eval.fct_daily_funnel_base

# 1) smoke: 12-scenario subset, runs on every push (cost-bounded)
python -m helios.eval.runner --smoke
```

```
# 2) full: all 50 scenarios, runs on PRs to main
python -m helios.eval.runner --full

# 3) render the results table + per-scenario JSON artifacts
python -m helios.eval.report --run-dir eval/history/$(date +%Y%m%d-%H%M)

# 4) seed the committed baseline from your first green full run
cp eval/history/<run>/summary.json eval/baselines/main.json
```

CI invokes the same entrypoints - smoke on push, full on PR to `main` - then compares against `eval/baselines/main.json`.

Tests to run

```
# scorer unit tests (deterministic; recorded fixtures, no live BigQuery)
pytest helios/eval/scorers/tests -v

# golden decomposition check reused here: mix=-0.0018, rate=0, interaction=0
pytest helios/eval/scorers/tests/test_decomposition.py::test_mape_zero_on_golden -v

# end-to-end smoke (drives the FSM against perturbed data)
python -m helios.eval.runner --smoke
```

Asserts: scorers return correct sub-scores on fixtures; `decomposition.py` yields MAPE 0 when the predicted split equals the analytic golden; `hallucination.py` returns a non-zero rate for any AST node not in the registry; the smoke run produces a `summary.json` with all seven metrics populated and every gate evaluated.

Success criteria

- Full run prints **root-cause top-1** ≥ 0.85 while the naive baseline ("largest absolute segment delta", Bible section 20.5) scores **~0.45** on the same scenarios - the headline near-doubling.
- **Hallucination rate** $= 0.000$ (hard gate; any non-zero fails CI regardless of accuracy).
- Decomposition MAPE ≤ 0.10 ; detection F1 ≥ 0.85 ; dollar-at-risk error ≤ 0.15 ; faithfulness ≥ 0.95 .
- Total scanned bytes per full run **under 5 GiB** (the harness dry_runs every scenario first and aborts if the per-run budget is exceeded).
- CI gate is green: PR fails if any threshold is breached OR top-1 regresses more than `regression_tolerance` (0.02) vs `eval/baselines/main.json`.
- Each scenario emits a per-scenario JSON (predicted vs label, every sub-score) under `eval/history/` for debugging.

Common mistakes

- **Grading on live (unlabeled) data** instead of injected synthetic labels - the public sample's real anomalies have no known root cause, so you cannot compute accuracy. Always grade against `helios_eval.labels`, never the public source.
- **No naive baseline to beat** - without the 45% baseline computed on the *same* scenarios, "85%" is meaningless. The baseline must run in the harness, not be a remembered constant.
- **Eval not wired into CI** -> silent regressions. The benchmark must be a *required* check on any PR touching `models/`, `semantic/`, `agents/`, or `eval/`.
- **Hallucination gate not hard-zero** - setting `hallucination_rate_max` above 0 defeats grounding-over-generation; keep it `0.00` and AST-check emitted SQL against the registry + GA4 schema.
- **Filename drift** - `mcp_servers.yaml` and `hallucination.py` must point at `models/semantic/semantic_layer.yaml` (not `semantic_models.yaml`); a stale path makes the AST check pass everything.
- **Cost blowup** - forgetting to dry_run every scenario, or letting the harness `SELECT *` over the perturbed shards. Cap bytes and abort over budget; the smoke subset (12) keeps push-time CI cheap.
- **Expecting byte-identical LLM output** - do not diff prose. Seed stats (`rng_seed 1729`), freeze the dbt model SHA + registry hash, and grade the *aggregate* metrics with tolerance.
- **Building M10 before M9 is green** - the eval harness is the agent layer's real grade; if the FSM isn't producing PASS findings with significance + dollars, every scorer reads garbage.

M11 - Autonomy & Depth (L3 EXIT)

Objective

Make Helios genuinely autonomous and statistically deeper: run on a schedule with no human in the loop, add forecast-residual anomaly detection plus cohort/RFM behavioral segmentation feeding the Diagnose hypothesis tree, expand the Critic to its full four-axis refutation battery, and add a secondary drill-down interface. The product's heartbeat is the **scheduled autonomous run** (conversation is a secondary surface). Exit when time-to-diagnosis holds **< 5 min/run on schedule**, accuracy holds **>= 85% across all canonical dimensions**, and memory-driven learning is demonstrable (a later run recognizes "seen before / already fixed / known-seasonal"). See **Bible section 23.3** (Phase 2 - Top-1% Undergrad / L3).

Files to create

Path	Copy from
<code>helios/scheduler.py</code>	Bible section 23.3 (scheduler endpoint: a single bounded run, idempotent, writes <code>run_state</code>)
<code>deploy/scheduler.yaml</code> or <code>deploy/crontab</code>	Cloud Scheduler job (HTTP target) or cron line invoking <code>python -m helios.scheduler</code>
<code>helios/mcp/stats.py</code> (extend)	MCP_ARCHITECTURE section 6.3 - add <code>forecast</code> (prophet/pmdarima), <code>cohort_retention</code> , <code>rfm_segment</code>
<code>helios/agents/monitor.py</code> (extend)	AGENT_ARCHITECTURE section 6.1 - Monitor uses <code>forecast</code> residuals (expected-vs-actual) for detection
<code>helios/agents/diagnose.py</code> (extend)	AGENT_ARCHITECTURE section 6.4 - feed <code>cohort_retention</code> / <code>rfm_segment</code> slices into the hypothesis tree
<code>helios/agents/critic.py</code> (extend)	AGENT_ARCHITECTURE section 6.5 - full battery: mix-shift confound, insufficient sample, seasonality, data quality
<code>helios/drilldown.py</code>	Bible section 23.3 - secondary conversational drill-down over governed metrics (read-only, semantic-mcp only)
<code>macros/channel_group.sql</code> (harden)	DBT_GUIDE section 2 - session-scoped <code>event_params</code> source/medium first, <code>traffic_source</code> first-touch fallback

The full Critic battery is already specified in AGENT_ARCHITECTURE section 6.5 - copy its four refutations exactly: (1) re-run `decompose_change` and DROP if `dominant = mix` ; (2) re-`significance_test` on an adjacent slice and DOWNGRADE if `p > alpha` or `n` small; (3) `recall_prior` + `seasonality_calendar` and DROP/DOWNGRADE if within `expected_mag` of a calendar entry overlapping the window; (4) data-quality confound probe (NULL spikes, `transaction_id` duplication, late shard) -> DROP if explained. Memory tools (`recall_prior` , `seasonality_calendar`) come from M8.

Commands to run

```
# 1) wire the new stats tools and re-validate the registry compiles clean
python scripts/validate_semantic.py          # 0 dangling references
python -m helios.mcp.stats                   # forecast / cohort_retention / rfm_segment
    expose

# 2) run one scheduled invocation by hand (must finish <5 min, write run_state)
python -m helios.scheduler --window last_28d

# 3a) Cloud Scheduler (GCP): daily 06:00 UTC, OIDC-authed HTTP target
gcloud scheduler jobs create http helios-daily \
    --schedule="0 6 * * *" --time-zone="UTC" \
```

```

--uri="$HELIOS_RUN_URL" --http-method=POST \
--oidc-service-account-email="helios-wh@${PROJECT}.iam.gserviceaccount.com"

# 3b) or local cron (POSIX): same daily cadence
# crontab -e → 0 6 * * * cd /opt/helios && .venv/bin/python -m helios.scheduler

# 4) drill-down interface (secondary surface, read-only)
python -m helios.drilldown --ask "checkout_to_purchase_rate by device_category last_14d"

```

Tests to run

```

# forecast residual + behavioral segmentation tool tests
pytest helios/mcp/tests/test_stats.py -k "forecast or cohort or rfm" -v

# full Critic battery: each axis DROPS/DOWNGRADES the right confound
pytest helios/agents/tests/test_critic.py -v
# asserts: pure mix labeled as rate → DROP; small-n → DOWNGRADE;
# known-seasonal move within expected_mag → DROP; NULL-spike confound → DROP;
# a clean finding → PASS

# accuracy holds across all canonical dimensions (full eval re-run)
python -m helios.eval.runner --full

# scheduled-run latency + idempotency
pytest helios/tests/test_scheduler.py -k "under_5min and idempotent" -v

```

Success criteria

- A scheduled invocation completes a full diagnosis in **< 5 min/run**, sustained across repeated scheduled fires, and writes `run_state` so re-fires are idempotent.
- Eval top-1 holds **>= 85%** when broken out **per canonical dimension** (device_category, channel_group, country, landing_page) - not just in aggregate.
- The Critic catches each adversarial axis: the seasonality_decoy bucket and data_quality bucket are NOT flagged as behavior changes (refuted), keeping detection precision high on the control/decoy scenarios.
- Forecast-residual Monitor reduces false positives on the no_anomaly_control bucket vs the M9 threshold detector (measurable drop in false-flag rate).
- **Memory-driven learning is demonstrable**: a second run over an overlapping window recalls a prior diagnosis (`recall_prior`) and either suppresses an already-fixed/known-seasonal issue or down-weights a root cause whose fix already won (`action_tracking.outcome = win`).

Common mistakes

- **Not seeding `seasonality_calendar`** (M8) -> the Critic is blind to known seasonality and flags the post-holiday dip in the decoy bucket as a real anomaly, tanking precision. Seed

BF2020/Dec/Jan before relying on the seasonality refutation.

- **suppression_list with no TTL** -> issues never resurface; a transient drop gets permanently muted. Every suppression carries an expiry.
- **Using event-level traffic_source as the session channel source** - it is USER first-touch, not session-scoped. Harden `channel_group_case()` to read session-scoped `event_params` source/medium first, falling back to `traffic_source` only when null; do not duplicate channel logic outside the single macro.
- **Not seeding the RNG** (`rng_seed 1729`) in forecast/cohort/RFM -> non-reproducible runs and flaky eval. All stats math is deterministic, seeded code in stats-mcp.
- **Making the LLM the scheduler/controller** - the scheduled entrypoint is deterministic Python that bounds the run (window, byte budget, refute rounds); the LLM only fills agent steps.
- **Skipping the full Critic battery** to "save time" - shipping mix-labeled-as-rate findings is exactly the failure the 85%-vs-45% claim is supposed to disprove; an unrefuted finding is not a finding.
- **Letting drill-down author raw SQL** - the secondary interface must go through semantic-mcp like every other path (G1/G5); it is read-only and bounded, never an ad-hoc SQL chatbot (that is the anti-product).

M12 - Productionization & Frontier (deferred)

Objective

Capture the post-L3 roadmap honestly: what production-hardening and frontier capabilities look like, and why each is explicitly deferred on *this* dataset. Do **not** build M12 to pass the L1-L3 gates - it is out of scope for the standalone build and listed so the reader knows the ceiling and the rationale. See **Bible section 23.4** (Phase 3 - Productionization) and **section 23.5** (Phase 4 - Frontier), with deferral rationale in section 23.7.

Files to create

None required for the standalone build. The items below are design targets, not deliverables for the playbook's MVP-to-L3 path. If pursued later, they extend existing artifacts (warehouse-mcp adapters, per-tenant `semantic_layer.yaml`, the stats-mcp causal module) rather than replacing them.

Phase 3 - Productionization (Bible section 23.4):

- **Multi-tenant isolation** - per-tenant semantic layer + per-tenant byte budgets so one tenant cannot scan another's data or blow a shared budget.

- **Warehouse-agnostic adapters** behind `warehouse-mcp` (Snowflake / Databricks / DuckDB) so the semantic layer remains the *only* SQL author regardless of dialect.
- **Streaming ingestion** of GA4 events (intraday) feeding `fct_daily_funnel` near-real-time, plus observability (per-agent latency, token cost, cache hit-rate).
- *Exit gate*: two warehouses pass **identical reconcile tests**; p95 run latency under SLA at N tenants.

Phase 4 - Frontier (Bible section 23.5):

- **Causal inference** - difference-in-differences, synthetic control, double-ML replacing correlational decomposition where the data supports it.
- **Auto-executed experiments** - push hypothesis cards to an experimentation platform, read back results, auto-update the backlog (closing the loop the Prescribe agent opens).
- **Multi-dataset** - join GA4 with CRM/cost data for true CAC/ROAS.
- *Exit gate*: a causal estimate **validated against a held-out randomized experiment**.

Commands to run

None. M12 has no commands in the standalone build path.

Tests to run

None for the standalone build. When pursued: the production exit gate is the *identical-reconcile-across-two-warehouses* test; the frontier exit gate is the *causal-estimate-vs-held-out-experiment* validation.

Success criteria

- The roadmap is documented and the deferrals are justified - not silently dropped. The honest constraints (Bible section 23.7) are:
 - **True causal inference** is deferred: the public GA4 sample has **no experiment assignment**, so L1-L3 ship *correlational* decomposition with explicit confound caveats from the Critic. Do not claim causality you cannot support.
 - **Real-time streaming** is deferred: a 3-month static sample gains nothing from intraday latency; batch is sufficient.
 - **True customer LTV** and **cross-device identity** are deferred: the ~3-month window is too short for lifetime curves (report 30/60-day proxy retention only), and `user_id` is almost always NULL so identity stitching is unsound.
 - **ML-based attribution** is deferred: it needs session-scoped channel + cost data the sample lacks; use GA4 default channel grouping until then.

Common mistakes

- **Over-claiming causality** - presenting correlational decomposition as a causal estimate. The Critic's job (and the brief's caveats) is to keep claims honest until Phase 4 data exists.
- **Building frontier features to hit L2/L3 gates** - the 85%-vs-45% claim is won by grounding + decomposition + the Critic + the eval harness, not by uplift modeling. Do not gold-plate.
- **Hardcoding BigQuery in agent/stats code** - if production-hardening is ever pursued, the *only* SQL author must remain the semantic layer; leaking dialect-specific SQL into agents breaks the warehouse-agnostic exit gate before it starts.
- **Promising LTV / cross-device on this data** - both are unsound on the ~3-month, mostly-NULL- `user_id` sample; state the proxy (30/60-day retention) and move on.